CASE STUDY DOCUMENT

Airline Ticket Booking System

— .NET / ASP.NET Core Edition —

SkyBooker

Search. Book. Fly. Effortlessly.

Version 1.0 | 2026 | .NET 8 / ASP.NET Core / Entity Framework Core

Platform Conversion Note

This document is the .NET / ASP.NET Core edition of the SkyBooker Airline Ticket Booking System Case Study. All architectural patterns, services, entities, and business logic are identical to the original Java/Spring Boot version. The following table summarises the key technology and terminology mappings applied throughout this document.

Java / Spring Boot	Equivalent .NET / ASP.NET Core
Java Class	C# Class
String / int / boolean / double / long	string / int / bool / decimal / long
LocalDateTime / LocalDate	DateTime / DateOnly
Optional<T>	T? (nullable reference type)
List<T> / IList<T>	IList<T> / List<T>
Map<K,V>	Dictionary<K,V>
Spring Boot Application	ASP.NET Core Web API / MVC Application
Spring MVC (@Controller)	ASP.NET Core MVC Controller (inherits Controller)
@RestController	[ApiController] with ControllerBase
ResponseEntity<T>	IActionResult / ActionResult<T>
Spring Data JPA (Repository)	Entity Framework Core (DbContext + IRepository<T>)
@Autowired (DI)	Constructor Injection via IServiceCollection (Program.cs)
@Service / @Component	Registered in DI container: builder.Services.AddScoped<>()
@Scheduled	IHostedService / BackgroundService
Spring Security + JWT	ASP.NET Core Authentication + JWT Bearer Middleware
Spring Security OAuth2	Microsoft.AspNetCore.Authentication.Google (OAuth2)
application.properties	appsettings.json
@Entity / @Table	[Table] attribute + EF Core DbSet<T>
Spring Boot Test / JUnit	xUnit / NUnit / MSTest + WebApplicationFactory<T>
Maven / Gradle	NuGet Package Manager + .csproj
Tomcat (embedded)	Kestrel (embedded, cross-platform)
Java package (e.g., com.skybooker.auth)	C# namespace (e.g., SkyBooker.Auth)
ServiceImpl (class)	Service (class implementing I-prefixed interface)
AuthResource / FlightResource	AuthController / FlightController (inherits ControllerBase)
JavaMailSender	IEmailSender (ASP.NET Core Identity) or MailKit / SendGrid SDK

Java / Spring Boot	Equivalent .NET / ASP.NET Core
SLF4J / Logback	Microsoft.Extensions.Logging (ILogger<T>)
RabbitMQ (Spring AMQP)	MassTransit + RabbitMQ or Azure Service Bus SDK
Redis (Spring Data Redis)	StackExchange.Redis or Microsoft.Extensions.Caching.StackExchangeRedis
iText 7 (PDF)	QuestPDF or iText 7 for .NET (itext7 NuGet package)
ZXing (QR code)	ZXing.Net (NuGet)
Docker / Kubernetes	Docker + Kubernetes (unchanged — platform-agnostic)
GitHub Actions CI/CD	GitHub Actions with dotnet build / dotnet test / Docker push

1. Synopsis

SkyBooker is a full-stack Airline Ticket Booking System inspired by MakeMyTrip and GoIbibo, built on .NET 8 and ASP.NET Core. It connects passengers with airlines, enabling them to search for one-way and round-trip flights, compare fare classes, select seats on an interactive seat map, enter passenger details, add ancillary services (meal preferences, extra baggage), and complete the booking with secure online payment — all from a unified platform.

The system supports four primary roles: Guests (who can search and browse flights without logging in), Passengers/Customers (who book, manage, and cancel tickets), Airline Staff (who manage flight schedules, seat inventory, and passenger manifests), and Platform Administrators (who oversee the entire ecosystem including airlines, airports, users, and analytics). Each role has a purpose-built dashboard rendered by ASP.NET Core MVC Razor views.

SkyBooker is architected as a microservices system using ASP.NET Core Web API projects for each service, with Entity Framework Core (EF Core) replacing Spring Data JPA, constructor-based dependency injection replacing Spring @Autowired, IHostedService/BackgroundService replacing Spring @Scheduled, and Microsoft.AspNetCore.Authentication.JwtBearer replacing Spring Security JWT — all following Clean Architecture principles with interface-first service contracts.

2. Detailed Requirements

2.1 General Requirements

- Guests can search and browse flights without logging in.
- Users must register or log in (email/password or Google OAuth 2.0) to book tickets.
- Role-based access control using ASP.NET Core Authorization Policies: Guest, Passenger, Airline Staff, and Admin.
- All users can update their profile and travel document details.
- In-app, email, and SMS notifications are dispatched for booking confirmation, check-in reminders, flight delays, and gate changes.
- Admin panel provides complete platform management including airline approvals, airport management, and analytics.

2.2 Passenger Requirements

- Register and log in via email/password or Google OAuth (via Microsoft.AspNetCore.Authentication.Google).
- Update profile including full name, phone number, passport number, and nationality.
- Search one-way flights by origin airport, destination airport, date, and number of passengers.
- Search round-trip flights specifying both departure and return dates.
- Filter search results by price range, airline, departure time, arrival time, number of stops, and seat class.

- View detailed flight information: airline, aircraft type, duration, stops, fare breakdown by class (Economy, Business, First).
- Select a fare class and view the interactive seat map for the chosen flight.
- Select available seats from the seat map (window, aisle, middle, extra legroom).
- Enter passenger details for each traveller: title, first/last name, date of birth, gender, passport number, nationality, and expiry date.
- Add meal preference (Veg/Non-Veg/Jain/Vegan) per passenger.
- Add extra baggage allowance (in kg) to the booking as a paid ancillary service.
- Review a complete booking summary with itemised fare breakdown before confirming.
- Make payment via credit/debit card, UPI, net banking, or wallet.
- Receive a booking confirmation email and SMS with the PNR code immediately after successful payment.
- Download the e-ticket and boarding pass as a PDF using QuestPDF or iText 7 for .NET.
- View all upcoming and past bookings on the My Bookings dashboard.
- Retrieve a booking by PNR code without logging in.
- Cancel a booking and initiate a refund (subject to the airline's cancellation policy).
- Perform web check-in (24 hours before departure) and re-select seat if needed.
- Receive automated check-in reminder notifications 24 hours before departure via BackgroundService.
- Receive flight delay, cancellation, and gate change alerts in real time.

2.3 Airline Staff Requirements

- Register as an airline operator and link to an approved airline entity on the platform.
- Add new flight schedules: flight number, origin, destination, departure/arrival times, aircraft type, and total seats.
- Edit existing flight schedules and update estimated times.
- Update flight status: On-Time, Delayed, Cancelled, Departed, or Arrived.
- Configure the seat map for a flight: add seats with class, row, column, features (window/aisle/extra legroom), and price multiplier.
- View the complete passenger manifest for any flight with passenger names, seat numbers, meal preferences, and check-in status.
- View flight-level revenue analytics: total bookings, revenue by fare class, seat utilisation.
- Send targeted flight delay or gate change alerts to all booked passengers on a specific flight.

2.4 Admin Requirements

- View and manage all user accounts: suspend, reactivate, or permanently delete.
- Create and manage airline profiles (IATA code, country, logo) and activate/deactivate airlines.
- Create and manage airport profiles (IATA/ICAO codes, city, country, timezone, GPS coordinates).
- View all bookings platform-wide with full lifecycle status and payment details.
- View all payment transactions and initiate refunds where applicable.

- Access platform analytics: total bookings, revenue, most popular routes, busiest airports, airline performance.
- Generate revenue reports for financial reconciliation and airline payouts.
- Send broadcast notifications to all users or by role (passengers, airline staff).
- View audit logs of all significant platform actions.

2.5 Booking Lifecycle Requirements

- Booking statuses: PENDING (payment not completed), CONFIRMED (payment successful), CANCELLED (by user or airline), COMPLETED (flight departed), NO_SHOW.
- A unique PNR (Passenger Name Record) code is generated at booking creation using a 6-character alphanumeric format.
- Seat reservation is held for 15 minutes during the booking process using EF Core optimistic concurrency (RowVersion / ConcurrencyToken); if payment is not completed, seats are auto-released by BackgroundService.
- Fare calculation includes base fare + taxes (GST/fuel surcharge) + ancillary add-ons (baggage, meal), itemised in a FareSummary record/DTO.
- Web check-in opens 24 hours before departure and closes 1 hour before scheduled departure time.

2.6 Payment Requirements

- Passengers can pay via credit/debit card, UPI, net banking, or platform wallet.
- Payment processing is handled by Razorpay .NET SDK or Stripe.net NuGet package with webhook callback.
- Each payment record stores amount, currency, mode, transaction ID, gateway response, and timestamps.
- On successful payment webhook, the booking status transitions to CONFIRMED and an e-ticket is generated asynchronously.
- Refunds are processed through the same gateway and credited to the original payment mode within 5-7 working days.
- Payment receipts are downloadable as PDF from the booking detail page.

2.7 Notification Requirements

- Notifications dispatched for: booking confirmed (email + SMS + app), check-in open reminder (24h before), flight delay/cancellation (push + email), gate change (push), boarding reminder (1h before).
- Each notification carries the RelatedBookingId for deep-linking to the booking detail page.
- Multi-channel dispatch: in-app notification centre, email via MailKit/SendGrid .NET SDK, and SMS via Twilio .NET SDK.
- Airline staff can manually trigger flight delay or gate change alerts for their flights.
- Automated BackgroundService sends check-in reminders 24 hours before departure.

3. Use Case Diagram

The diagram below captures all actors and use cases within the SkyBooker .NET platform — spanning flight search, booking lifecycle, seat selection, passenger management, payment, web check-in, notifications, airline schedule management, and administrative oversight.

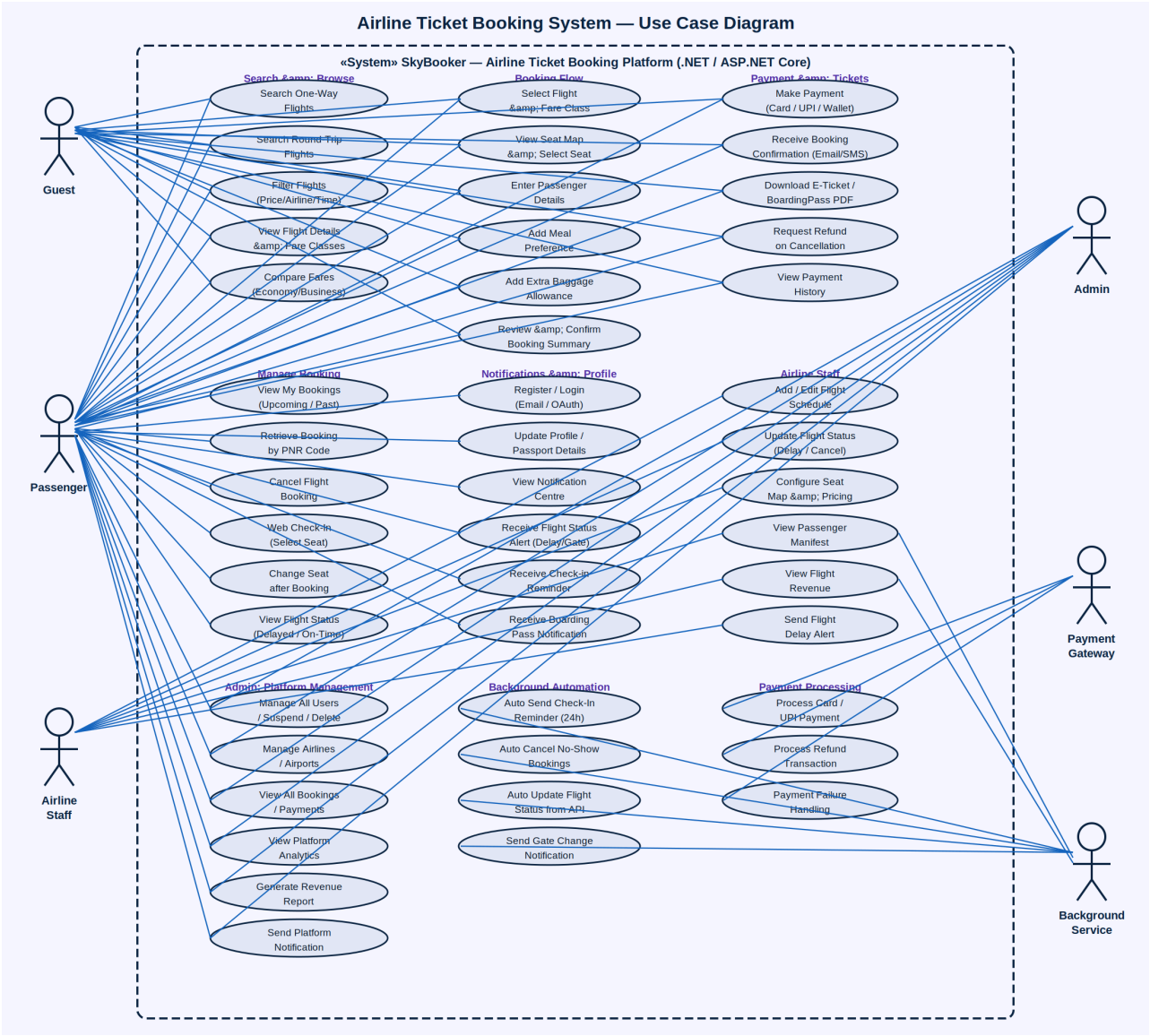

Figure 1: SkyBooker Airline Ticket Booking System (.NET) — Complete Use Case Diagram

3.1 Actors

Actor	Description
Guest	Unauthenticated visitor who can search and browse flights without logging in

Actor	Description
Passenger	Registered user who books, manages, and cancels flight tickets
Airline Staff	Airline operator who manages flight schedules, seat inventory, and passenger manifests
Admin	Platform administrator managing airlines, airports, users, and platform analytics
Payment Gateway	External service (Razorpay .NET SDK / Stripe.net) processing card, UPI, and net banking payments
Background Service	IHostedService / BackgroundService handling check-in reminders, seat hold release, and flight status sync

3.2 Use Case Summary

Use Case	Actor(s)	Description
Search One-Way Flights	Guest, Passenger	Find available flights between two airports on a specified departure date for N passengers
Search Round-Trip Flights	Guest, Passenger	Find outbound and return flight pairs for a specified origin, destination, and two dates
Filter Flights	Guest, Passenger	Narrow results by price, airline, departure time, stops, and seat class
View Flight Details	Guest, Passenger	See fare breakdown by class, aircraft type, duration, and available seats per class
Compare Fares	Guest, Passenger	Side-by-side comparison of Economy, Business, and First Class prices and inclusions
Register / Login	Passenger	Create account or authenticate via email/password or Google OAuth
Update Profile	Passenger	Change name, phone, passport number, nationality, and travel document details
Select Flight & Fare Class	Passenger	Choose a flight and a specific fare class to proceed to seat selection
View Seat Map	Passenger	Open the interactive seat map and see available, occupied, and held seats by class
Select Seat	Passenger	Reserve a preferred seat (window, aisle, extra legroom) for each passenger
Enter Passenger Details	Passenger	Fill in title, name, date of birth, gender, passport, and nationality for each traveller
Add Meal Preference	Passenger	Select a meal type (Veg/Non-Veg/Jain/Vegan) for each passenger
Add Extra Baggage	Passenger	Purchase additional baggage allowance in kg as a paid ancillary add-on

Use Case	Actor(s)	Description
Review Booking Summary	Passenger	See itemised fare breakdown (base + taxes + ancillaries) before confirming
Make Payment	Passenger, Gateway	Pay via card, UPI, net banking, or wallet through the integrated payment gateway
Receive Booking Confirmation	Passenger	Get PNR code, e-ticket, and confirmation via email and SMS immediately after payment
Download E-Ticket / PDF	Passenger	Download formatted e-ticket and boarding pass PDF generated by QuestPDF / iText 7
View My Bookings	Passenger	Browse all upcoming and past bookings with status and PNR details
Retrieve Booking by PNR	Guest, Passenger	Look up any booking using its 6-character PNR code without logging in
Cancel Booking	Passenger	Cancel a confirmed booking; trigger refund based on the airline's cancellation policy
Request Refund	Passenger, Gateway	Initiate refund transaction to the original payment mode via Razorpay/Stripe webhook
View Payment History	Passenger	See all payment transactions with amounts, modes, and status
Web Check-In	Passenger	Complete online check-in within the 24h-1h pre-departure window
Change Seat after Booking	Passenger	Re-select a seat from the seat map after booking and before check-in closes
View Flight Status	Guest, Passenger	See real-time status (On-Time, Delayed, Cancelled, Departed) for any flight
View Notification Centre	Passenger	Browse all in-app alerts with read/unread status and booking deep-links
Receive Flight Status Alert	Passenger, BG Service	In-app and push notification for flight delays, cancellations, and gate changes
Receive Check-In Reminder	Passenger, BG Service	Automated BackgroundService alert 24 hours before scheduled departure
Receive Boarding Pass Alert	Passenger	Notification with boarding pass ready to download after successful check-in
Add / Edit Flight Schedule	Airline Staff	Create or update a flight with number, route, times, aircraft type, and total seats
Update Flight Status	Airline Staff	Mark a flight as Delayed, Cancelled, Departed, or Arrived in real time
Configure Seat Map	Airline Staff	Define seat layout with classes, rows, columns, features, and price multipliers
View Passenger Manifest	Airline Staff	Access the full passenger list for a flight with seat, meal, and check-in status

Use Case	Actor(s)	Description
View Flight Revenue	Airline Staff	See total bookings, revenue by fare class, and seat utilisation for any flight
Send Flight Delay Alert	Airline Staff	Manually trigger a delay or gate change notification to all booked passengers
Manage All Users	Admin	View, suspend, reactivate, or permanently delete any user account
Manage Airlines	Admin	Create, edit, activate, or deactivate airline profiles with IATA code and logo
Manage Airports	Admin	Create and update airport profiles with IATA/ICAO codes, city, country, and timezone
View All Bookings	Admin	Monitor every booking platform-wide with full lifecycle status
View Platform Analytics	Admin	Access total bookings, revenue, popular routes, and airline performance metrics
Generate Revenue Report	Admin	Export financial data for airline payouts and platform commission reconciliation
Send Platform Notification	Admin	Broadcast an alert to all users or by role (passengers/airline staff)
Auto-Release Seat Hold	BG Service	BackgroundService releases seat hold if payment is not completed within 15 minutes
Auto Cancel No-Show	BG Service	Transition un-checked-in bookings to NO_SHOW status after gate closure
Auto Update Flight Status	BG Service	Sync flight departure/arrival status from airline API at scheduled intervals

4. Class Diagrams — All Services (.NET)

SkyBooker .NET follows the same layered microservices architecture as the EShoppingZone reference system, adapted for C# and ASP.NET Core. Each service comprises four layers: Entity/Model (domain model with EF Core attributes), Repository Interface (IRepository<T> with EF Core DbContext), Service Interface (I-prefixed contract) + Service Implementation, and API Controller (inherits ControllerBase, returns IActionResult). Dependency injection is configured in Program.cs via builder.Services.AddScoped<>.

4.1 Auth / User-Service

The Auth/User-Service manages all user identity operations. JWT authentication is configured via AddAuthentication().AddJwtBearer() in Program.cs, with the token secret stored in appsettings.json (or Azure Key Vault in production). Google OAuth2 is added via AddAuthentication().AddGoogle(). The Role property (PASSENGER/AIRLINE_STAFF/ADMIN) is used in [Authorize(Roles = "Admin")] attributes on protected controller endpoints. The User entity is mapped to the database via EF Core and a UsersDbContext.

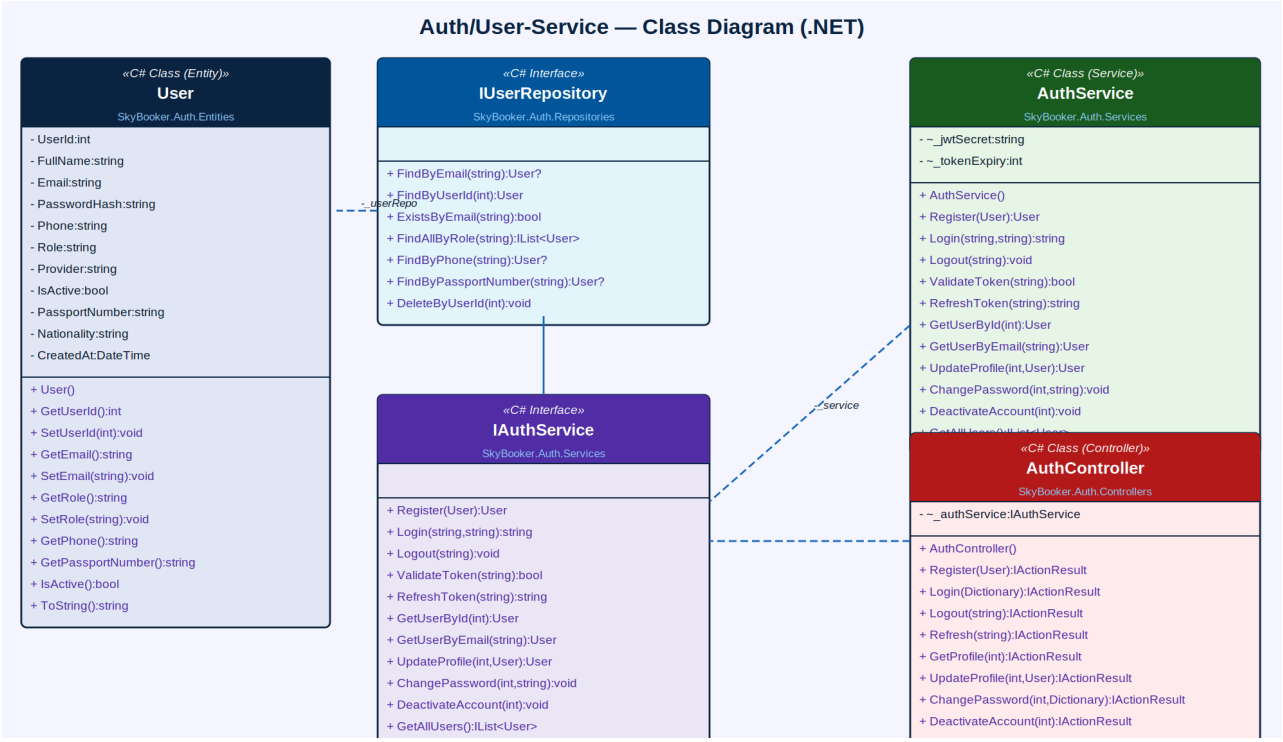

Figure 2: Auth/User-Service — Class Diagram (.NET / C#)

Component	Type	Responsibility
User	C# Class (EF Core Entity)	Properties: UserId, FullName, Email, PasswordHash, Phone, Role (PASSENGER/STAFF/ADMIN), Provider, IsActive, PassportNumber, Nationality, CreatedAt (all PascalCase C# properties with EF Core [Column] attributes)
IUserRepository	C# Interface	FindByEmail(), FindByUserId(), ExistsByEmail(), FindAllByRole(), FindByPhone(), FindByPassportNumber(), DeleteByUserId() — implemented by UserRepository using EF Core DbContext
AuthService	C# Class	Implements IAuthService; injected via constructor; uses IUserRepository, IConfiguration (for JWT secret), and PasswordHasher<User> from ASP.NET Core Identity

Component	Type	Responsibility
IAuthService	C# Interface	Declares Register(), Login(), Logout(), ValidateToken(), RefreshToken(), GetUserById(), UpdateProfile(), ChangePassword(), DeactivateAccount(), GetAllUsers()
AuthController	C# Class (ControllerBase)	[ApiController][Route("api/auth")] — Exposes POST /register, POST /login, POST /logout, POST /refresh, GET/PUT /profile, PUT /password, DELETE /deactivate, GET /users

4.2 Flight-Service

The Flight-Service manages flight schedule inventory. In .NET, the Flight entity is mapped via EF Core with [Index] attributes on FlightNumber (unique) and composite indexes on OriginAirportCode + DestinationAirportCode + DepartureTime for fast search queries. The AvailableSeats counter is decremented atomically using EF Core ExecuteUpdateAsync with optimistic concurrency. SearchRoundTrip returns a Dictionary<string, IList<Flight>> with 'outbound' and 'return' keys.

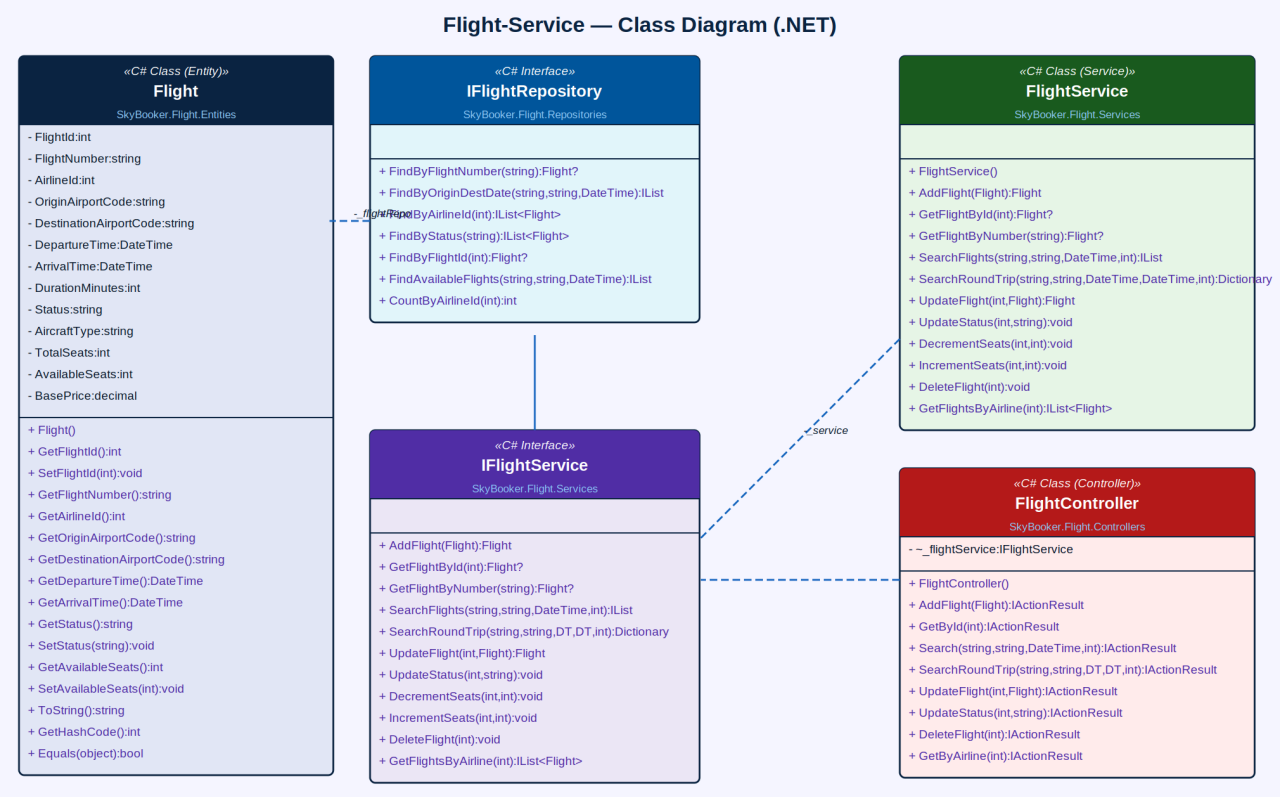

Figure 3: Flight-Service — Class Diagram (.NET / C#)

Component	Type	Key Attributes / Methods
Flight	C# Class (EF Core Entity)	FlightId, FlightNumber, AirlineId, OriginAirportCode, DestinationAirportCode, DepartureTime (DateTime), ArrivalTime (DateTime), DurationMinutes, Status (enum or const string), AircraftType, TotalSeats, AvailableSeats, BasePrice (decimal)
IFlightRepository	C# Interface	FindByFlightNumber(), FindByOriginDestDate(), FindByAirlineId(), FindByStatus(), FindByFlightId(), FindAvailableFlights(), CountByAirlineId() — all return Task<T> for async/await pattern
FlightService	C# Class	AddFlight(), GetFlightById(), GetFlightByNumber(), SearchFlights(), SearchRoundTrip(), UpdateFlight(), UpdateStatus(), DecrementSeats(),

Component	Type	Key Attributes / Methods
		IncrementSeats(), DeleteFlight(), GetFlightsByAirline()
IFlightService	C# Interface	Declares all flight CRUD, one-way/round-trip search, status update, and atomic seat counter management — all methods return Task<T> for async support
FlightController	C# Class (ControllerBase)	[ApiController][Route("api/flights")] — POST (add), GET (by id/number/airline), GET search (one-way/round-trip), PUT (update/status), DELETE

4.3 Seat / Aircraft-Service

The Seat/Aircraft-Service manages seat inventory. Each Seat entity uses an EF Core [ConcurrencyToken] attribute on the Status field to implement optimistic concurrency for seat holds — if two users attempt to hold the same seat simultaneously, EF Core throws a DbUpdateConcurrencyException which is caught in the SeatService and surfaced as a 409 Conflict response. The seat hold TTL is enforced by a BackgroundService scanning for seats with Status=HELD and HeldSince older than 15 minutes.

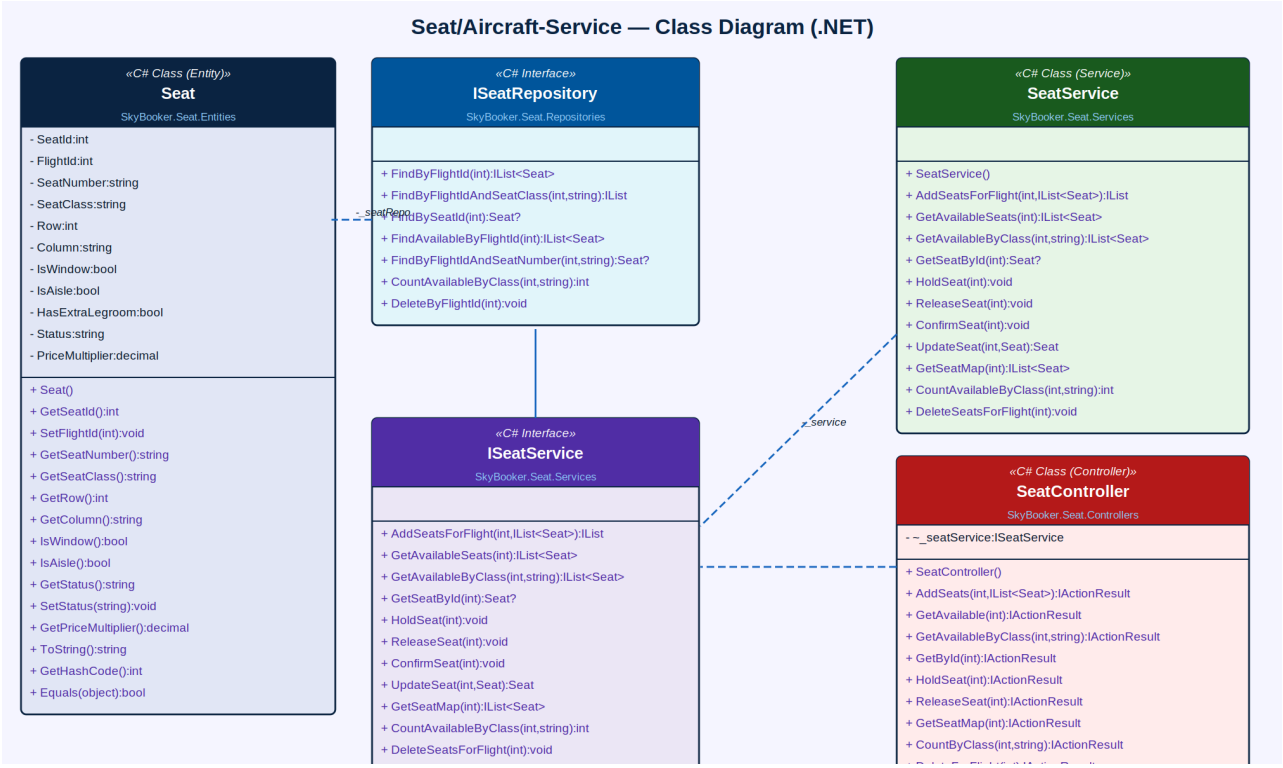

Figure 4: Seat/Aircraft-Service — Class Diagram (.NET / C#)

Component	Type	Key Attributes / Methods
Seat	C# Class (EF Core Entity)	SeatId, FlightId, SeatNumber (e.g. 12A), SeatClass (Economy/Business/First), Row, Column, IsWindow (bool), IsAisle (bool), HasExtraLegroom (bool), Status ([ConcurrencyToken] AVAILABLE/HELD/CONFIRMED/BLOCKED), PriceMultiplier (decimal)
ISeatRepository	C# Interface	FindByFlightId(), FindByFlightIdAndSeatClass(), FindBySeatId(), FindAvailableByFlightId(), FindByFlightIdAndSeatNumber(), CountAvailableByClass(), DeleteByFlightId() — async via Task<T>
SeatService	C# Class	AddSeatsForFlight(), GetAvailableSeats(), GetAvailableByClass(), GetSeatById(), HoldSeat(), ReleaseSeat(), ConfirmSeat(), UpdateSeat(),

Component	Type	Key Attributes / Methods
		GetSeatMap(), CountAvailableByClass(), DeleteSeatsForFlight()
ISeatService	C# Interface	Declares seat creation, availability queries, hold/release/confirm lifecycle (with DbUpdateConcurrencyException handling), seat map retrieval, and count operations
SeatController	C# Class (ControllerBase)	[ApiController][Route("api/seats")] — POST (addSeats), GET (available/by class/map/id), PUT (hold/release/update), GET count, DELETE (for flight)

4.4 Booking-Service

The Booking-Service orchestrates the booking workflow. In the .NET implementation, CreateBooking uses EF Core transactions (await _context.Database.BeginTransactionAsync()) to atomically persist the Booking record, update Seat statuses to HELD, and decrement Flight.AvailableSeats. FareSummary is a C# record type (immutable DTO) containing BaseFare, Taxes, AncillaryCost, and TotalFare properties. PNR generation uses Guid.NewGuid() truncated and uppercased to 6 characters with collision detection.

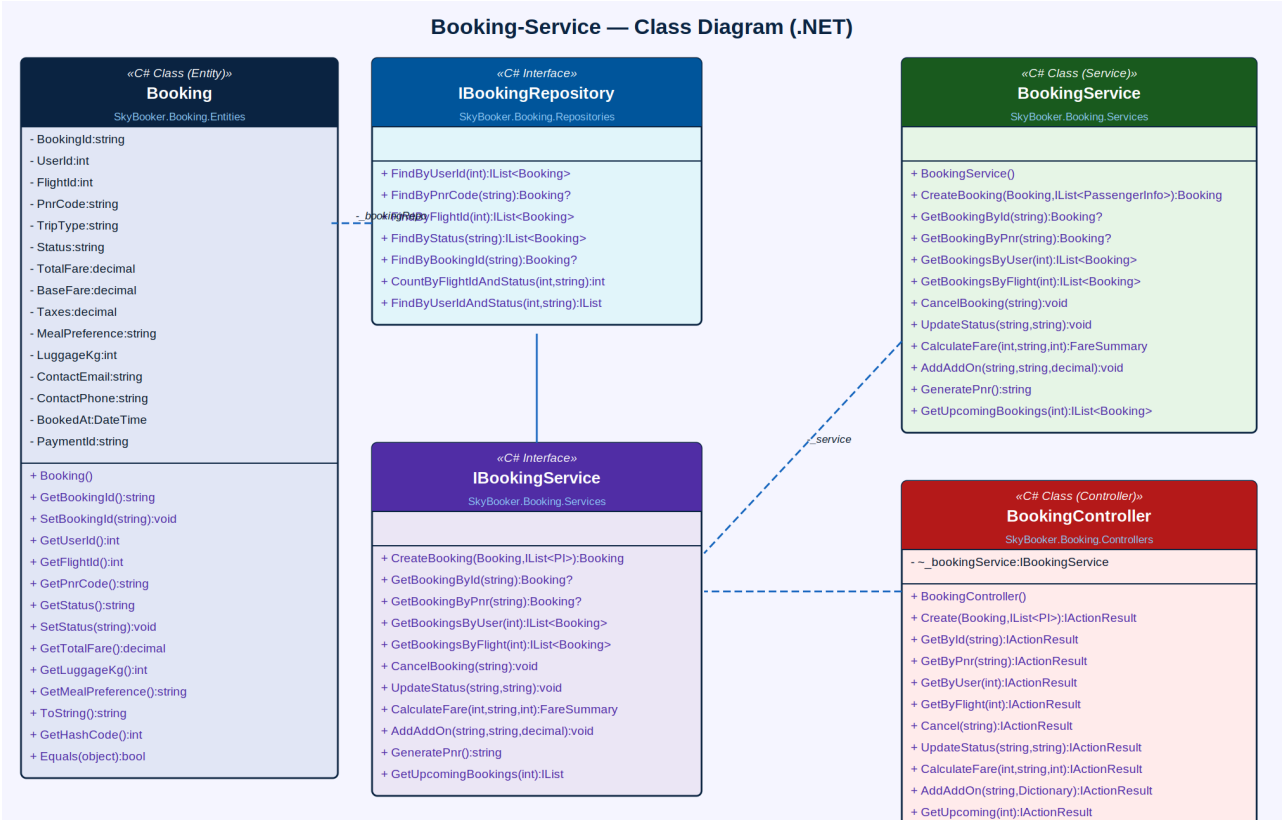

Figure 5: Booking-Service — Class Diagram (.NET / C#)

Component	Type	Key Attributes / Methods
Booking	C# Class (EF Core Entity)	BookingId (string/Guid), UserId, FlightId, PnrCode (6-char), TripType (ONE_WAY/ROUND_TRIP), Status enum, TotalFare/BaseFare/Taxes (decimal), MealPreference, LuggageKg, ContactEmail, ContactPhone, BookedAt (DateTime), PaymentId
IBookingRepository	C# Interface	FindByUserId(), FindByPnrCode(), FindByFlightId(), FindByStatus(), FindByBookingId(), CountByFlightIdAndStatus(), FindByUserIdAndStatus() — all Task<T> async
BookingService	C# Class	CreateBooking() uses EF Core transaction scope; GetBookingById(), GetBookingByPnr(), GetBookingsByUser(), GetBookingsByFlight(),

Component	Type	Key Attributes / Methods
		CancelBooking(), UpdateStatus(), CalculateFare() returns FareSummary record, AddAddOn(), GeneratePnr(), GetUpcomingBookings()
IBookingService	C# Interface	Declares booking creation with IList<PassengerInfo>, PNR retrieval, cancellation, fare calculation, add-on management, and upcoming-bookings query — all Task<T>
BookingController	C# Class (ControllerBase)	[ApiController][Route("api/bookings")] — POST (create), GET (by id/pnr/user/flight/upcoming), PUT (cancel/status), GET calculateFare, POST addAddOn

4.5 Passenger-Service

The Passenger-Service stores detailed traveller information. In .NET, PassengerInfo is validated using FluentValidation (FluentValidation.AspNetCore NuGet) with rules for passport expiry (must be future date), age eligibility by PassengerType (ADULT 12+, CHILD 2-11, INFANT under 2), and mandatory fields. TicketNumber generation uses a formatted string: '{airline-code}{flight-number}-{random-6-digit}'. After web check-in, the SeatId and SeatNumber properties are updated via AssignSeat().

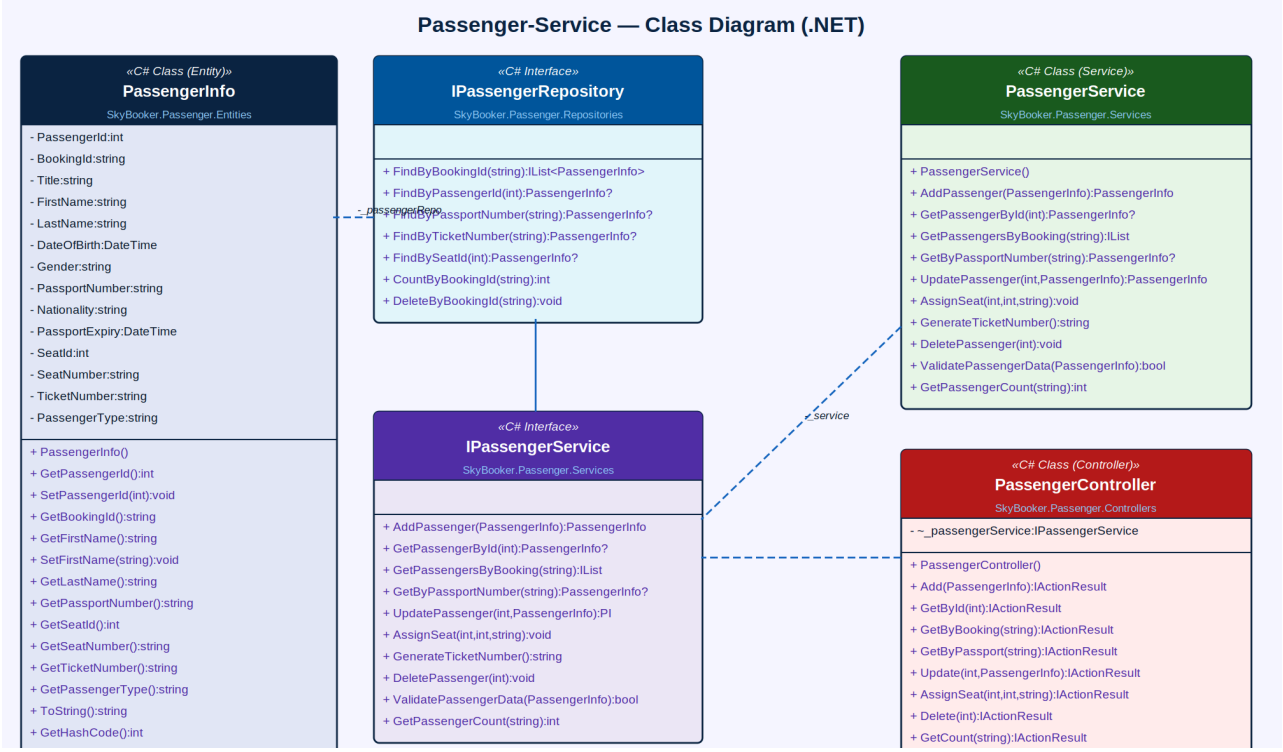

Figure 6: Passenger-Service — Class Diagram (.NET / C#)

Component	Type	Key Attributes / Methods
PassengerInfo	C# Class (EF Core Entity)	PassengerId, BookingId (string FK), Title, FirstName, LastName, DateOfBirth (DateTime), Gender, PassportNumber, Nationality, PassportExpiry (DateTime), SeatId, SeatNumber, TicketNumber (generated), PassengerType (ADULT/CHILD/INFANT)
IPassengerRepository	C# Interface	FindByBookingId(), FindByPassengerId(), FindByPassportNumber(), FindByTicketNumber(), FindBySeatId(), CountByBookingId(), DeleteByBookingId() — async Task<T>
PassengerService	C# Class	AddPassenger() with FluentValidation; GetPassengerById(), GetPassengersByBooking(), GetByPassportNumber(), UpdatePassenger(), AssignSeat(), GenerateTicketNumber(), DeletePassenger(), ValidatePassengerData(), GetPassengerCount()

Component	Type	Key Attributes / Methods
IPassengerService	C# Interface	Declares all passenger CRUD, seat assignment, ticket number generation, validation, and count — all Task<T>
PassengerController	C# Class (ControllerBase)	[ApiController][Route("api/passengers")] — POST (add), GET (by id/booking/passport), PUT (update/assignSeat), DELETE, GET count

4.6 Payment-Service

The Payment-Service handles all financial transactions using Razorpay .NET SDK or Stripe.net. InitiatePayment creates a pending payment record and returns a Razorpay/Stripe order/session object. A webhook endpoint (POST /api/payments/webhook) receives the gateway callback, verifies the HMAC signature, and calls ProcessPayment to update status and trigger BookingService.UpdateStatus(). GenerateReceipt uses QuestPDF to produce a formatted PDF receipt returned as FileContentResult.

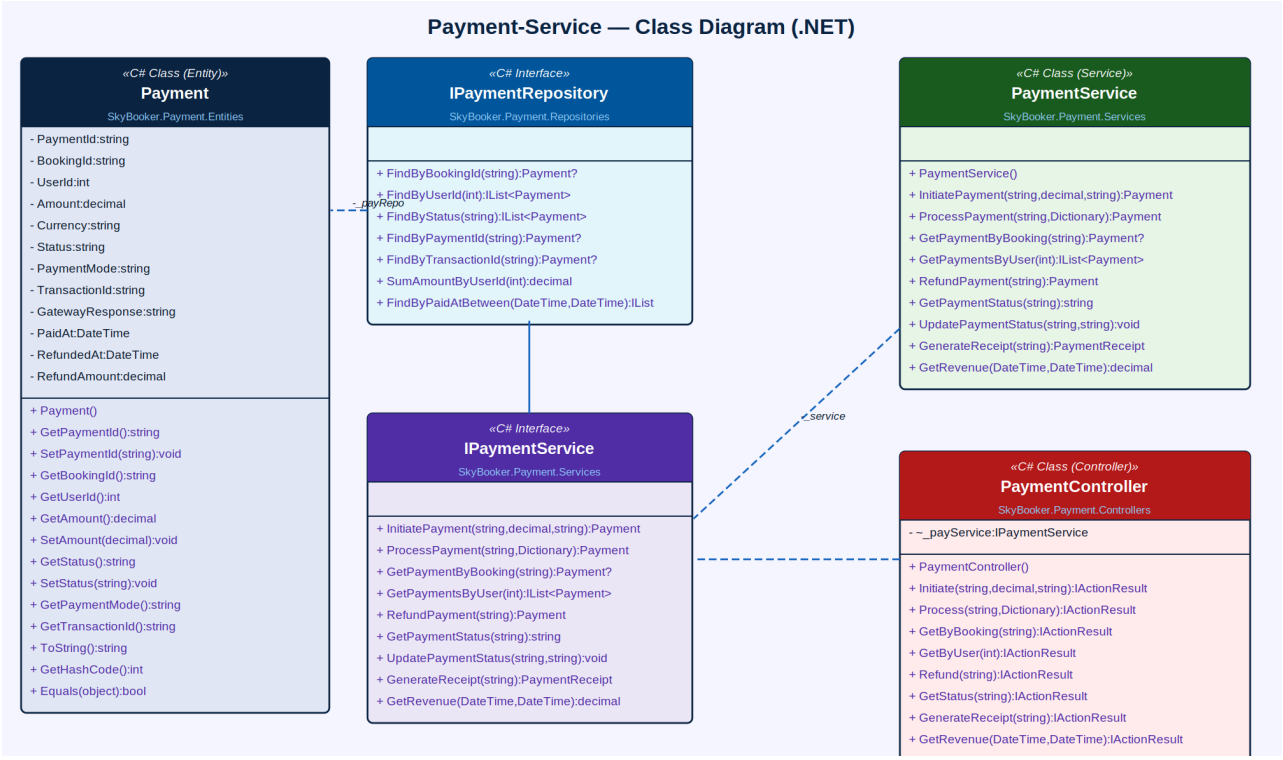

Figure 7: Payment-Service — Class Diagram (.NET / C#)

Component	Type	Key Attributes / Methods
Payment	C# Class (EF Core Entity)	PaymentId (string/Guid), BookingId, UserId, Amount (decimal), Currency, Status (PENDING/PAID/FAILED/REFUNDED), PaymentMode (CARD/UPI/NETBANKING/WALLET), TransactionId, GatewayResponse, PaidAt (DateTime?), RefundedAt (DateTime?), RefundAmount (decimal)
IPaymentRepository	C# Interface	FindByBookingId(), FindByUserId(), FindByStatus(), FindByPaymentId(), FindByTransactionId(), SumAmountByUserId(), FindByPaidAtBetween() — all Task<T>
PaymentService	C# Class	InitiatePayment(), ProcessPayment() (webhook handler with HMAC verification), GetPaymentByBooking(), GetPaymentsByUser(),

Component	Type	Key Attributes / Methods
		RefundPayment(), GetPaymentStatus(), UpdatePaymentStatus(), GenerateReceipt() returns FileContentResult, GetRevenue()
IPaymentService	C# Interface	Declares payment initiation, gateway callback processing, refund, status query, QuestPDF receipt generation, and revenue aggregation — all Task<T>
PaymentController	C# Class (ControllerBase)	[ApiController][Route("api/payments")] — POST (initiate/process-webhook/refund), GET (by booking/user/status/receipt), GET revenue(DateTime,DateTime)

4.7 Notification-Service

The Notification-Service dispatches multi-channel alerts. In .NET, email is sent via MailKit (MimeKit/MailKit NuGet) or the SendGrid .NET SDK. SMS is dispatched via Twilio .NET SDK (Twilio NuGet). Push notifications use Firebase Admin SDK for .NET. The SendBookingConfirmation method attaches the e-ticket PDF (generated by QuestPDF) as an email attachment using MimeKit's MimePart API.

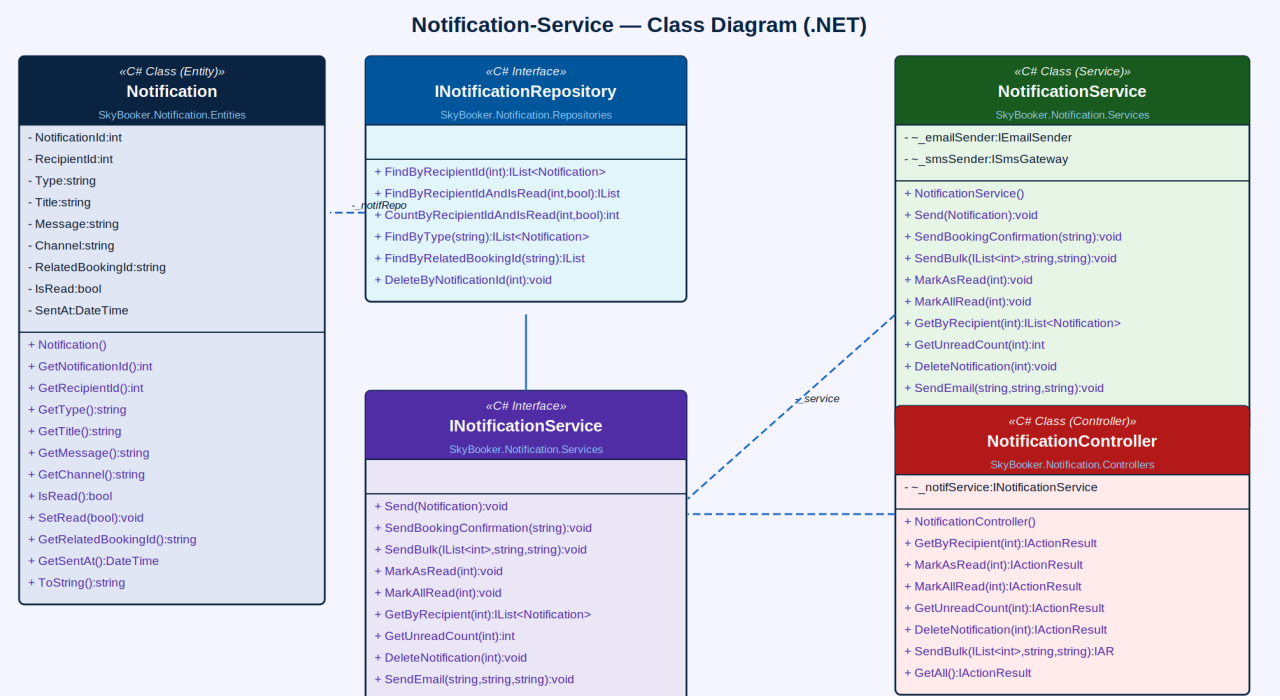

Figure 8: Notification-Service — Class Diagram (.NET / C#)

Component	Type	Key Attributes / Methods
Notification	C# Class (EF Core Entity)	NotificationId, RecipientId, Type (BOOKING_CONFIRMED/FLIGHT_DELAY/GATE_CHANGE/CHECKIN_REMINDER/BOARDING/CANCELLATION), Title, Message, Channel (APP/EMAIL/SMS), RelatedBookingId (string), IsRead (bool), SentAt (DateTime)
INotificationRepository	C# Interface	FindByRecipientId(), FindByRecipientIdAndIsRead(), CountByRecipientIdAndIsRead(), FindByType(), FindByRelatedBookingId(), DeleteByNotificationId() — all Task<T>
NotificationService	C# Class	Send(), SendBookingConfirmation() with PDF attachment via MimeKit, SendBulk(), MarkAsRead(), MarkAllRead(), GetByRecipient(), GetUnreadCount(), DeleteNotification(), SendEmail() via MailKit/SendGrid, SendSMS() via Twilio SDK

Component	Type	Key Attributes / Methods
INotificationService	C# Interface	Declares single/bulk notification dispatch, booking confirmation with PDF, email/SMS channel dispatch, read-state management, and deletion — all Task<T>
NotificationController	C# Class (ControllerBase)	[ApiController][Route("api/notifications")] — GetByRecipient, MarkAsRead, MarkAllRead, GetUnreadCount, Delete, SendBulk, GetAll

4.8 Airline & Airport-Service

The Airline & Airport-Service manages master data for airlines and airports. In .NET, IataCode properties have EF Core [Index(IsUnique = true)] attributes. The SearchAirports method uses EF Core LIKE query (EF.Functions.Like) for autocomplete. Both Airline and Airport entities are in the same AirlineDbContext; the Airport entity has a navigation property linking to Airline via a many-to-many relationship through an AirlineAirport join table.

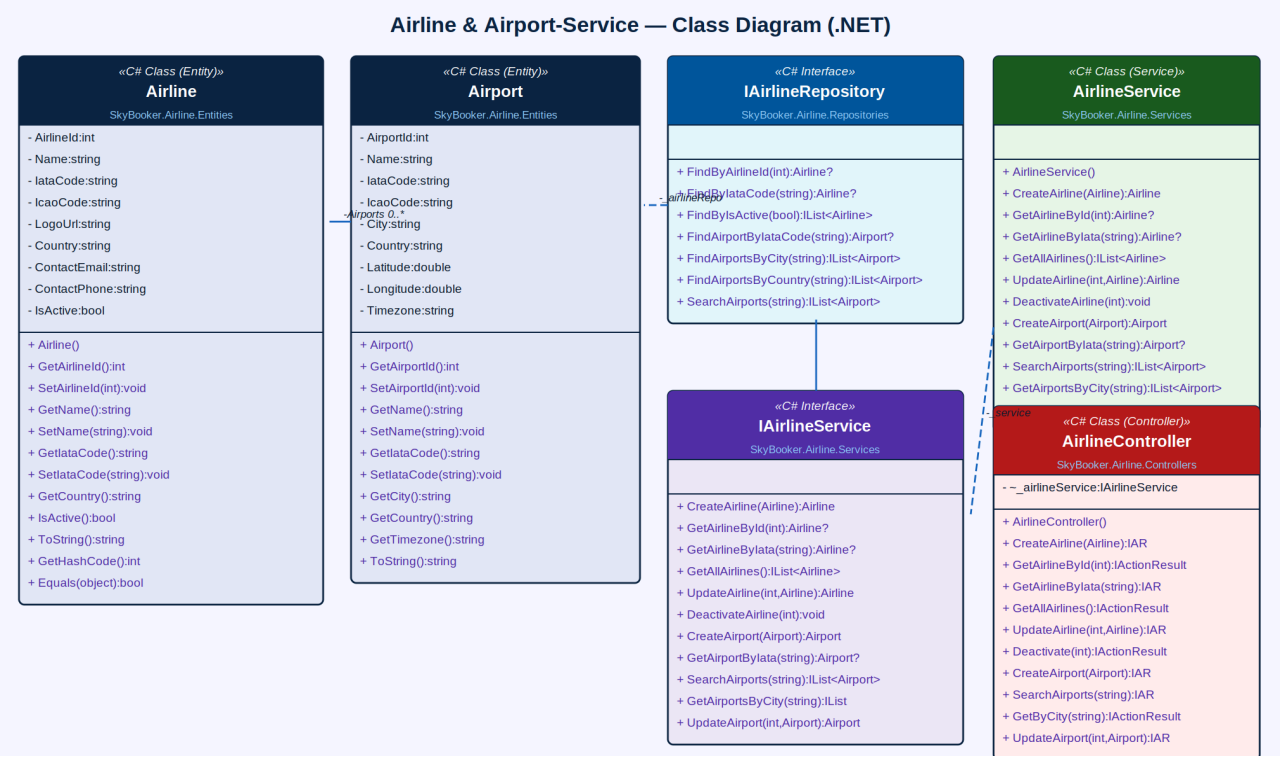

Figure 9: Airline & Airport-Service — Class Diagram (.NET / C#)

Component	Type	Key Attributes / Methods
Airline	C# Class (EF Core Entity)	AirlineId, Name, IataCode ([Index(IsUnique=true)]), IcaoCode, LogoUrl, Country, ContactEmail, ContactPhone, IsActive (bool)
Airport	C# Class (EF Core Entity)	AirportId, Name, IataCode ([Index(IsUnique=true)]), IcaoCode, City, Country, Latitude (double), Longitude (double), Timezone — many-to-many with Airline
IAirlineRepository	C# Interface	FindByAirlineId(), FindByIataCode(), FindByIsActive(), FindAirportByIataCode(), FindAirportsByCity(), FindAirportsByCountry(), SearchAirports() using EF.Functions.Like() — all Task<T>
AirlineService	C# Class	CreateAirline(), GetAirlineById(), GetAirlineByIata(), GetAllAirlines(), UpdateAirline(), DeactivateAirline(), CreateAirport(),

Component	Type	Key Attributes / Methods
		GetAirportByIata(), SearchAirports(), GetAirportsByCity(), UpdateAirport()
IAirlineService	C# Interface	Declares all airline and airport CRUD, IATA-based lookup, EF.Functions.Like search, and deactivation — all Task<T>
AirlineController	C# Class (ControllerBase)	[ApiController][Route("api/airlines")] and [Route("api/airports")] — POST, GET (by id/iata/city/search/all), PUT (update/deactivate)

4.9 MVC Controllers (Razor / Web Layer)

The MVC web layer uses ASP.NET Core MVC with Razor Views (.cshtml). Three controllers inherit from Controller (not ControllerBase) to return ViewResult and access ViewBag/ViewData. All service dependencies are injected via the constructor from the DI container configured in Program.cs. The IActionResult return type used throughout all controllers supports ViewResult, RedirectToActionResult, JsonResult, and FileContentResult.

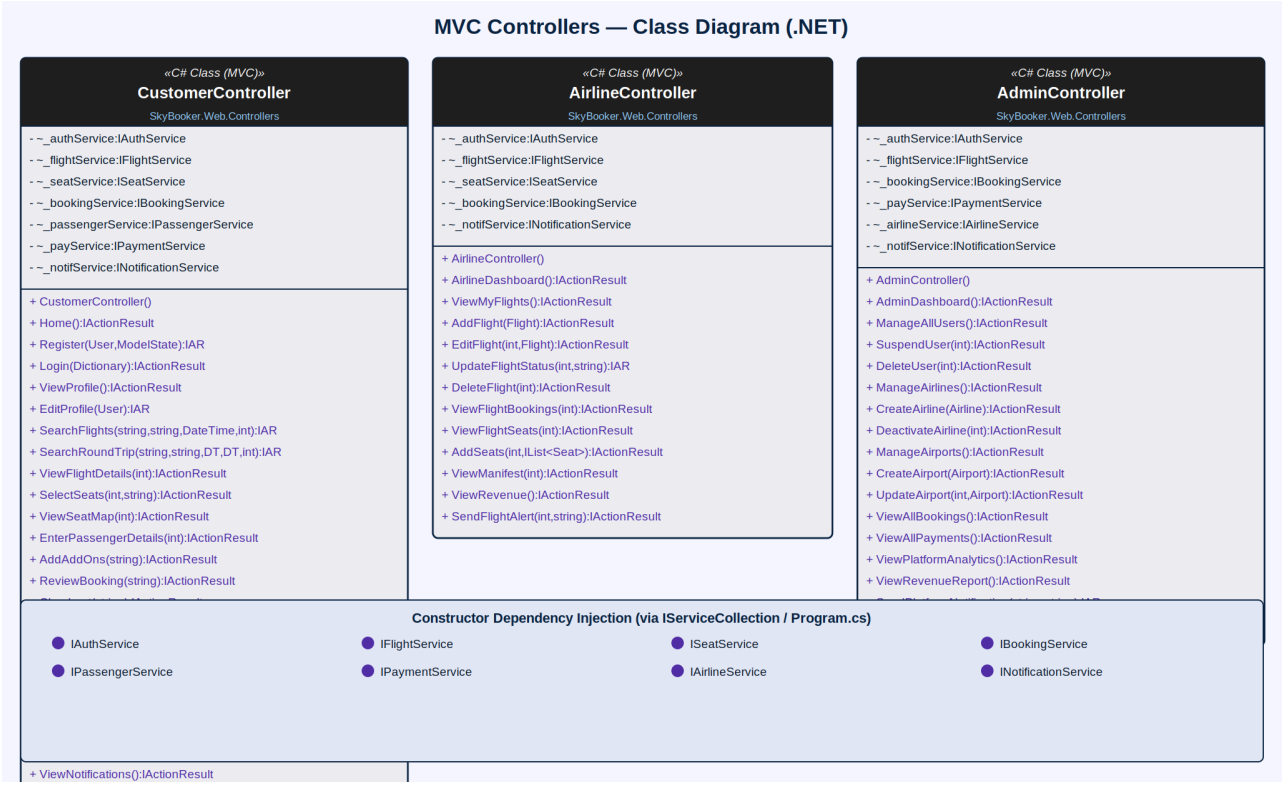

Figure 10: MVC Controllers — Class Diagram (.NET / ASP.NET Core)

Controller	Type	Key Methods
CustomerController	C# Class (Controller)	Home(), Register(), Login(), ViewProfile(), EditProfile(), SearchFlights(), SearchRoundTrip(), ViewFlightDetails(), SelectSeats(), ViewSeatMap(), EnterPassengerDetails(), AddAddOns(), ReviewBooking(), Checkout(), MakePayment(), BookingConfirmation(), ViewMyBookings(), ViewBookingDetail(), CancelBooking(), DownloadETicket(), WebCheckin(), ViewNotifications(), Logout()
AirlineController	C# Class (Controller)	AirlineDashboard(), ViewMyFlights(), AddFlight(), EditFlight(), UpdateFlightStatus(), DeleteFlight(), ViewFlightBookings(), ViewFlightSeats(), AddSeats(), ViewManifest(), ViewRevenue(), SendFlightAlert()

Controller	Type	Key Methods
AdminController	C# Class (Controller)	AdminDashboard(), ManageAllUsers(), SuspendUser(), DeleteUser(), ManageAirlines(), CreateAirline(), DeactivateAirline(), ManageAirports(), CreateAirport(), UpdateAirport(), ViewAllBookings(), ViewAllPayments(), ViewPlatformAnalytics(), ViewRevenueReport(), SendPlatformNotification(), ViewAuditLogs()

5. Microservices Architecture Overview (.NET)

SkyBooker .NET is structured as independently deployable ASP.NET Core Web API projects following Clean Architecture. Each project references its own EF Core DbContext and exposes a REST API. Synchronous inter-service calls use IHttpClientFactory (typed HttpClient); asynchronous messaging uses MassTransit + RabbitMQ. Background jobs use IHostedService or the Hangfire .NET library.

Microservice (.NET Project)	Namespace	Primary Domain
SkyBooker.Auth.API	SkyBooker.Auth	User accounts, JWT Bearer, Google OAuth2, passport/nationality details
SkyBooker.Flight.API	SkyBooker.Flight	Flight schedules, search (one-way/round-trip), status, EF Core seat counters
SkyBooker.Seat.API	SkyBooker.Seat	Seat map, hold/release/confirm with EF Core ConcurrencyToken, class-based pricing
SkyBooker.Booking.API	SkyBooker.Booking	Booking lifecycle, PNR generation, EF Core transactions, ancillary add-ons
SkyBooker.Passenger.API	SkyBooker.Passenger	Passenger details, FluentValidation, ticket numbers, seat assignment
SkyBooker.Payment.API	SkyBooker.Payment	Razorpay/Stripe.net integration, webhook processing, QuestPDF receipts
SkyBooker.Notification.API	SkyBooker.Notification	MailKit/SendGrid email, Twilio SMS, Firebase FCM push, in-app alerts
SkyBooker.Airline.API	SkyBooker.Airline	Airline and airport EF Core data, IATA codes, EF.Functions.Like search
SkyBooker.Web	SkyBooker.Web	ASP.NET Core MVC, Razor Views (.cshtml), Bootstrap 5, Razor Tag Helpers

6. Non-Functional Requirements

Category	Requirement
Performance	Flight search results must load within 2 seconds; IMemoryCache / IDistributedCache (StackExchange.Redis) caches popular route results with 5-minute sliding expiry
Scalability	Each ASP.NET Core Web API project independently scalable in Docker/Kubernetes; Kestrel handles concurrent connections; YARP reverse proxy for API gateway routing
Security	Passwords hashed with PasswordHasher<T> (PBKDF2 + HMAC-SHA256); JWT tokens expire in 24h; [Authorize] + Policy-based authorisation; HTTPS enforced; PCI-DSS via Razorpay/Stripe tokenisation
Availability	99.9% uptime SLA; /health endpoints via Microsoft.Extensions.Diagnostics.HealthChecks; Polly (Polly.Extensions.Http NuGet) for retry and circuit breaker on inter-service calls
Data Integrity	EF Core ConcurrencyToken on Seat.Status prevents double-booking; EF Core transactions (BeginTransactionAsync) ensure atomic booking creation
Seat Hold Expiry	IHostedService polls every 2 minutes for Seat.Status=HELD with HoldExpiry < DateTime.UtcNow and calls ReleaseSeat(); configurable expiry via appsettings.json
PNR Uniqueness	PNR codes are 6-character alphanumeric generated from Guid.NewGuid().ToString("N").Substring(0,6).ToUpper() with database unique constraint via [Index(IsUnique=true)]
Fare Accuracy	Dynamic pricing: base fare locked at time of HoldSeat() using EF Core snapshot read; IMemoryCache stores fare snapshot for 15 minutes per booking session
Audit Trail	All booking transitions, payment events, and admin actions logged via ILogger<T> and persisted to an AuditLog EF Core entity for compliance
Usability	Responsive Razor Views with Bootstrap 5; seat map rendered as SVG in Razor partial view; WCAG 2.1 AA with ARIA attributes on interactive seat grid

7. Proposed Technology Stack (.NET)

Layer	Technology / NuGet Package
Backend Framework	ASP.NET Core 8 (Web API + MVC); .NET 8 SDK; Kestrel embedded web server
Authentication	Microsoft.AspNetCore.Authentication.JwtBearer (JWT); Microsoft.AspNetCore.Authentication.Google (OAuth2); ASP.NET Core Identity (password hashing)
ORM / Data Access	Entity Framework Core 8 (Microsoft.EntityFrameworkCore); EF Core SQL Server or MySQL provider; EF Core Migrations for schema management
Database	SQL Server (primary, via Microsoft.EntityFrameworkCore.SqlServer) or MySQL (Pomelo.EntityFrameworkCore.MySql); StackExchange.Redis (distributed cache and session)
Validation	FluentValidation.AspNetCore NuGet — for PassengerInfo, booking, and registration model validation
Payment Gateway	Razorpay .NET SDK (Razorpay NuGet) or Stripe.net (Stripe.net NuGet); webhook HMAC-SHA256 signature verification
PDF Generation	QuestPDF (QuestPDF NuGet) for e-ticket and boarding pass PDF layout; iText7 (itext7 NuGet) as alternative
QR Code	ZXing.Net (ZXing.Net NuGet) for barcode/QR generation on boarding passes
Seat Map UI	SVG rendered in Razor partial views; optionally a React/TypeScript component with fetch API calls to SeatController
File Storage	Azure Blob Storage (Azure.Storage.Blobs NuGet) or AWS S3 (AWSSDK.S3) for e-ticket PDFs; pre-signed URLs via SAS tokens or pre-signed S3 URLs
Email	MailKit (MailKit NuGet) with SMTP; or SendGrid .NET SDK (SendGrid NuGet) via AWS SES SMTP relay
SMS	Twilio .NET SDK (Twilio NuGet) for booking confirmation and check-in reminder SMS
Push Notifications	Firebase Admin .NET SDK (FirebaseAdmin NuGet) for Android/iOS push via FCM
Messaging	MassTransit (MassTransit.RabbitMQ NuGet) for async notification fan-out, booking email queue, and flight status sync events
Background Jobs	IHostedService / BackgroundService for seat hold release (2-min polling), check-in reminders (hourly), no-show detection; or Hangfire (Hangfire.AspNetCore NuGet)
Resilience	Polly (Microsoft.Extensions.Http.Polly NuGet) for retry policies and circuit breakers on IHttpClientFactory typed clients
Frontend	Razor Views (.cshtml) with Bootstrap 5, jQuery, and Tag Helpers; or React.js SPA with fetch API to ASP.NET Core Web API endpoints

Layer	Technology / NuGet Package
API Gateway	YARP (Yarp.ReverseProxy NuGet) reverse proxy for routing, load balancing, and JWT validation at gateway level
Containerisation	Docker with multi-stage Dockerfile; Docker Compose for local development; Kubernetes for production autoscaling
API Documentation	Swashbuckle.AspNetCore (Swagger/OpenAPI 3.0) auto-generated from controller XML comments
Logging	Microsoft.Extensions.Logging (ILogger<T>) with Serilog (Serilog.AspNetCore NuGet) sink to Seq or Application Insights
Health Checks	Microsoft.Extensions.Diagnostics.HealthChecks + AspNetCore.HealthChecks.SqlServer + AspNetCore.HealthChecks.Redis
CI/CD	GitHub Actions: dotnet restore → dotnet build → dotnet test → docker build → push to ACR/ECR → kubectl rolling update

8. Glossary

Term	Definition
ASP.NET Core	Microsoft's open-source, cross-platform web framework for building Web APIs and MVC web applications on .NET 8
IActionResult	The return type of ASP.NET Core controller action methods; supports Ok(), NotFound(), BadRequest(), File(), Redirect() and other HTTP responses
ControllerBase	The base class for ASP.NET Core Web API controllers providing model binding, validation, and HTTP response helpers without Razor view support
Controller	The base class for ASP.NET Core MVC controllers that adds View() and Redirect() support for Razor view rendering on top of ControllerBase
Entity Framework Core	Microsoft's official ORM for .NET, mapping C# classes to database tables via DbContext and LINQ-to-SQL queries
DbContext	The EF Core database session class managing entity tracking, change detection, and database operations for a microservice
ConcurrencyToken	An EF Core attribute marking a property as an optimistic concurrency check field — throws DbUpdateConcurrencyException on conflicting updates
IHostedService	An ASP.NET Core interface for background tasks running in the web host's lifetime — used for seat hold release, reminders, and flight status sync
BackgroundService	An abstract base implementing IHostedService for long-running background loops, replacing Spring @Scheduled in the .NET ecosystem
IServiceCollection	The ASP.NET Core DI container builder in Program.cs; services are registered via builder.Services.AddScoped<IService, ServiceImpl>()
FluentValidation	A popular .NET library for strongly-typed validation rules defined in AbstractValidator<T> classes, replacing javax.validation annotations
MailKit	A cross-platform .NET email library (MimeKit/MailKit NuGet) providing SMTP/IMAP support for sending notification emails with attachments
QuestPDF	A modern .NET library for generating PDFs using a fluent C# layout API — used for e-ticket and boarding pass generation
MassTransit	A .NET open-source message broker abstraction library supporting RabbitMQ and Azure Service Bus for async inter-service communication
YARP	Yet Another Reverse Proxy — Microsoft's ASP.NET Core-based reverse proxy NuGet package used as API gateway for routing between microservices
Polly	A .NET resilience library (Microsoft.Extensions.Http.Polly) providing retry, circuit breaker, timeout, and bulkhead policies for IHttpClientFactory calls
PNR	Passenger Name Record — a unique 6-character alphanumeric booking reference generated via Guid.NewGuid().ToString("N").Substring(0,6).ToUpper()

Term	Definition
EF Core Migration	A code-first database schema versioning mechanism in EF Core (dotnet ef migrations add) replacing Flyway/Liquibase in the Java stack
Kestrel	The cross-platform embedded web server in ASP.NET Core, equivalent to the embedded Tomcat in Spring Boot
appsettings.json	The .NET configuration file replacing Spring Boot's application.properties; supports environment-specific overrides (appsettings.Production.json)